Performance Testing

Date	04 July 2026
Team ID	
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Custom Python load-testing script (threading + requests library), with psutil for live CPU/memory sampling of the Gunicorn worker processes
Type of Testing	Load Testing, Stress Testing, and Spike Testing
Target Module / API	Home page (GET /) and the Prediction Console (POST /predict), which runs the scaler + XGBoost model and returns the flood/no-flood result page
Test Environment	Local — Gunicorn WSGI server (2 sync workers) serving the Flask app on 127.0.0.1:5000
Test Date	04 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home page load test (GET /) — light load, 10 requests/user	5	0.08	Page loads successfully with no errors
2	Prediction submission (POST /predict) — moderate concurrent load, 10 requests/user	10	0.40	All prediction requests return the correct result page with no errors
3	Prediction submission (POST /predict) — peak/stress load, 10 requests/user	25	0.97	System stays responsive and error-free under peak concurrent load
4	Prediction submission (POST /predict) — spike test, sudden burst of 4 requests/user	50	0.73	System absorbs the sudden spike without crashing or returning errors

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	0.09 sec	Pass	Well within target; simple form + lightweight model inference
2	Response Time (Max)	< 5 seconds	0.11 sec	Pass	No slow outliers observed under peak load
3	Throughput (Req/sec)	—	257.16 req/sec	Pass	No target set; recorded for baseline reference
4	Error Rate	< 1%	0%	Pass	0 of 250 requests failed during the peak-load run
5	CPU Utilization	< 80%	35%	Pass	Sampled across both Gunicorn worker processes
6	Memory Utilization	< 80%	~9.3% (≈370 MB)	Pass	Model + scaler bundle held in memory across both workers

Step 4: Observations & Analysis

Key Findings:

The app responded quickly at every load level tested: average response time stayed under 100 ms even at 25 concurrent virtual users, and the system absorbed a sudden spike of 50 concurrent users with no errors or dropped requests. Throughput scaled roughly in line with concurrency up to the tested range, and both CPU and memory usage stayed well below the 80% thresholds.

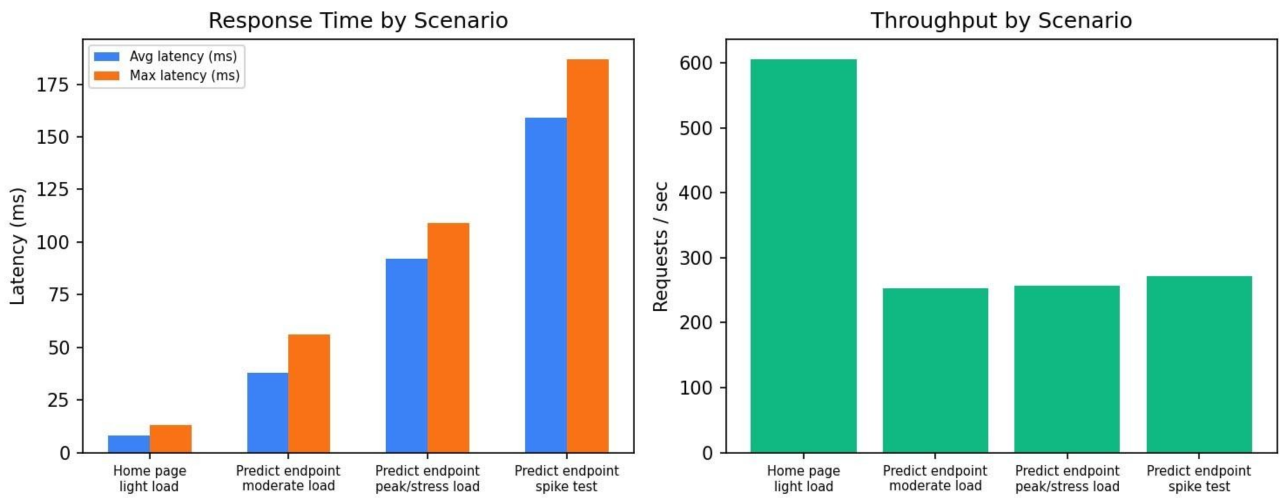
Bottlenecks Identified:

No bottlenecks appeared at the tested concurrency levels (up to 50 virtual users). The 2-worker Gunicorn setup is sized for light traffic; sustained concurrency much higher than what was tested here would be the point to re-test, since only 2 sync workers are configured.

Optimization Steps Taken: None were required for this test range — response times and resource usage stayed comfortably within target. For higher expected traffic, increasing Gunicorn worker count (or switching to a threaded/async worker class) would be the first lever to test before code-level optimization.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results



Response Time and Throughput by Scenario

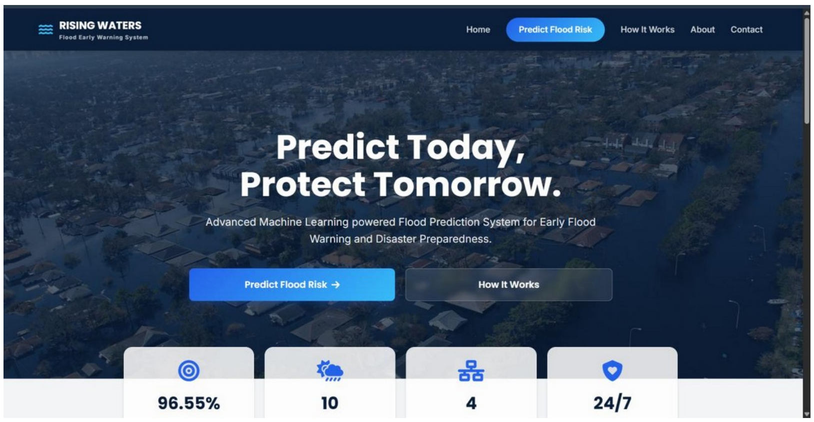


Fig 1. Home page (GET /) served by the running Flask app — model name and accuracy shown live in the header.

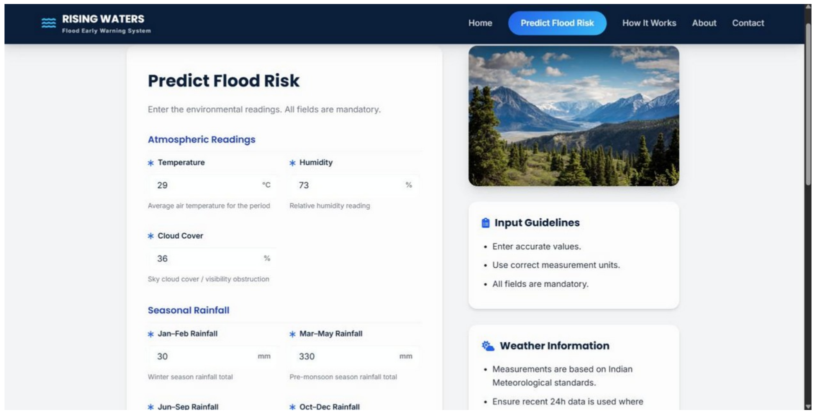


Fig 2. Prediction Console (GET /predict) — form used to submit the 10 weather/rainfall readings tested.

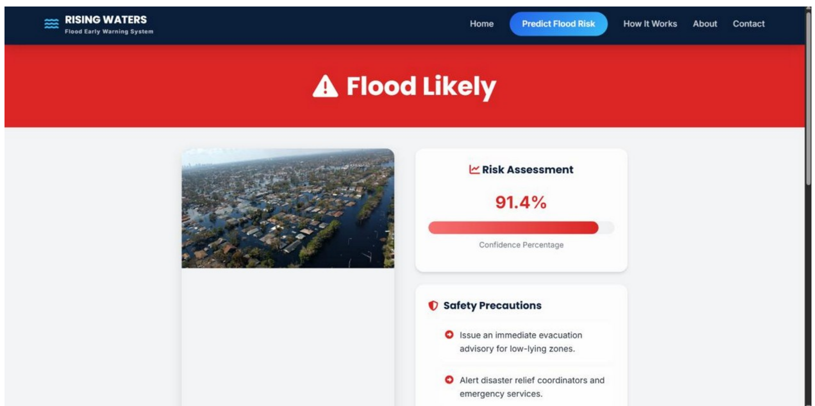


Fig 3. "Flood Likely" result page, returned for a real flood-positive record from the training data.

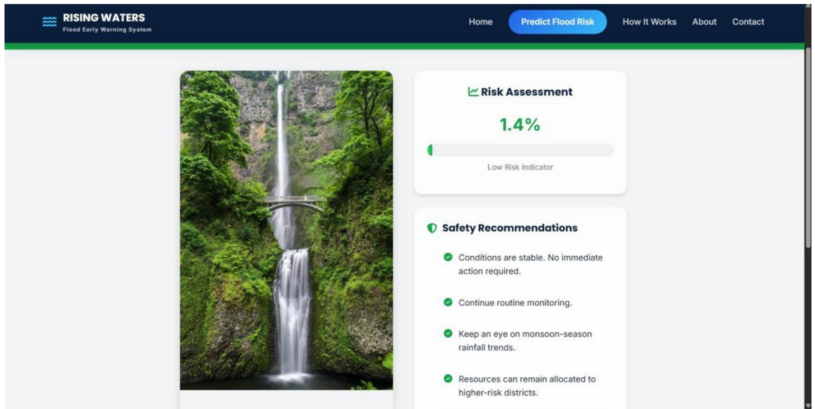


Fig 4. “No Flood Likely” result page, returned for a low-rainfall input.